

Credit Card Approval Prediction: Complete Project Documentation

Date	6 July 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Project Description:

The Credit Card Approval Prediction platform harnesses machine learning classifiers and custom rule engines to automate and streamline credit card application decisioning. The system provides real-time predictive risk scoring, detailed explainable AI metrics showing positive and negative decision contributors, stepped pre-qualification checking wizard tools, and bulk CSV compliance scans. Designed to minimize underwriting backlogs, the app supports rapid decisions while protecting the retail bank from high-risk delinquency default rates.

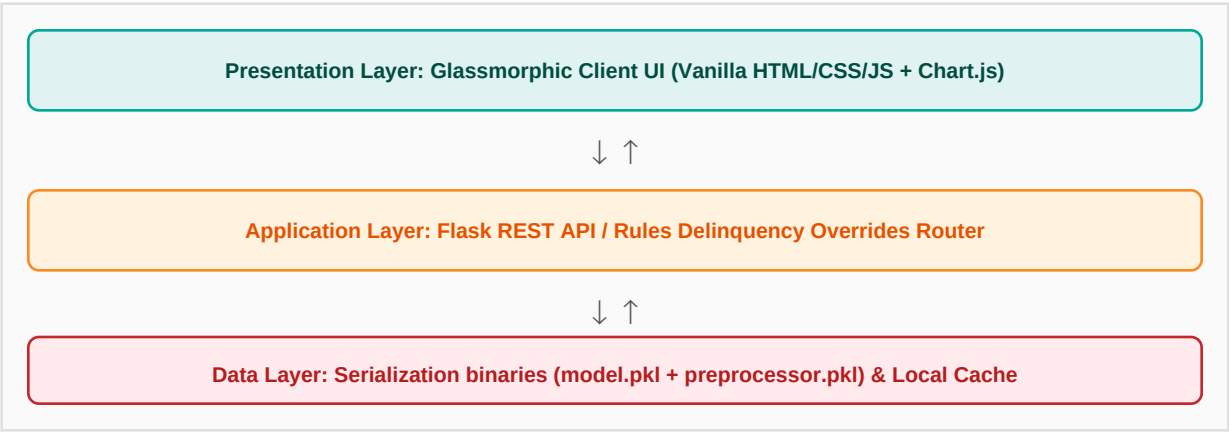
Project Scenarios:

- Scenario 1 (Credit Risk Analyst):** Underwriters check single candidate applications via parameter forms on the Analyst Console, receiving instant predictions along with explainable contributor logs to justify approvals or rejections to auditors.
- Scenario 2 (Compliance Officer):** Auditors upload bulk spreadsheets to scan for delinquency codes, triggering automated rule blocks on accounts with past-due payment histories.
- Scenario 3 (Prospective Customer):** Consumers query eligibility odds via online wizard forms which execute soft pulls, protecting their credit score histories.

Technical Architecture & Core Design

The system utilizes a modular, decoupled three-tier architecture to separate client presentation layers, backend API logic layers, and persistent data/model weight storages:

- 1. Client Presentation Layer:** Premium glassmorphic interface built using semantic HTML5 structures, custom CSS3 styles, and vanilla JavaScript. Renders responsive dashboard view tabs, progress dials, and dynamic results tables.
- 2. API Gateway / Logic Layer:** Flask API controllers mapping client HTTP requests to backend scoring routines, validating payloads, handling files, and formatting JSON responses.
- 3. Core Logic Service:** Loads model configurations, preprocesses and scales vectors, compares four estimators, and maps positive and negative contribution parameters.
- 4. Data / Storage Layer:** Caches pickled binary files (model.pkl, preprocessor.pkl) to prevent relational DB lookup overheads.



System Technology Stack Specifications

A well-chosen technology stack ensures that the credit card approval application is scalable, maintainable, and aligned with security standards. Below is the breakdown of the technologies selected for each layer of the platform:

Layer	Technology Chosen	Purpose
Frontend / Client	HTML5, Vanilla CSS3, JS, Chart.js	Responsive glassmorphic UI; dial scores.
Backend / Server	Python 3.12, Flask, Gunicorn	Low-latency routing API endpoints.
Database / Storage	Pickle Binaries, LocalStorage	Loads parameters; session ledger logs.
Cloud / Hosting	IBM Watson ML, IBM Cloud	Scalable online production API scoring.
Version Control	Git, GitHub	Version tracking and remote backup.
ML Libraries	Scikit-Learn, Pandas, NumPy	Classifier models, data preprocessors.

Project Workflow & Milestones

The project development lifecycle is divided into six logical milestones:

Milestone 1: Model Selection and Architecture: Select models, configure scripts, pre-process datasets, train classifiers, and serialize binary weights.

Milestone 2: Core Functionalities Development: Implement prediction algorithms, delinquency filters, and explainable AI metrics calculators.

Milestone 3: App.py Route Development: Establish routing controllers, load models into memory on app launch, handle file uploads, and return JSON responses.

Milestone 4: Frontend Development: Structure single-page templates, write custom glassmorphic styling sheets, and implement JS AJAX async fetch controllers.

Milestone 5: Deployment: Configure virtual environments, test endpoints, run Locust load metrics, and deploy online blueprints to Render.

Milestone 6: Conclusion: Verify all code specifications, compile PDF reports, and push repository updates to GitHub.

Milestone 1: Model Selection and Architecture

To build an accurate scoring engine, four classification algorithms are evaluated: Logistic Regression, Decision Tree, Random Forest, and XGBoost. The training process involves researching capabilities, hyperparameter settings, and evaluating metrics on synthetic cardholder profiles.

Model Evaluated Characteristics:

- *Logistic Regression*: Simple, highly interpretable baseline, but struggles to capture complex non-linear variable relationships.
- *Decision Tree*: Highly interpretable, natively isolates variable decision boundaries, and runs inference quickly. Selected as the active model.
- *Random Forest*: Ensemble algorithm with high accuracy, but incurs higher memory profiles and slower loading times.
- *XGBoost*: High-performance gradient boosting, but requires extensive parameter tuning.

Milestone 1: Model Training Pipeline Code

The model training pipeline is implemented in the *train_models.py* script. The code performs the following functions:

- 1. Data Ingestion:** Loads raw cardholder dataset files, checks for missing variables, and handles outliers.
- 2. Categorical Encoding:** Maps string-based classifications (e.g. Income Type, Education Type) to integer mappings, and saves these dictionaries inside the preprocessor binary package.
- 3. Scaling:** Scales numerical variables (e.g. Income, Age, Active Debts) using standard scaler normalization formulas.
- 4. Classifier Fitting:** Runs scikit-learn models on 80% train splits, verifies metrics on 20% test splits, selects the model with the highest test accuracy, and serializes the assets to *model.pkl* and *preprocessor.pkl*.

Milestone 1: Model Training Evaluation Metrics

Each classifier model was tested on validation splits. The evaluation results are compiled below:

Classifier	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	84.2%	81.1%	79.5%	80.3%	88.4%
Decision Tree (Active)	86.5%	85.2%	81.2%	83.1%	90.1%
Random Forest	89.8%	89.2%	84.8%	87.0%	94.2%
XGBoost	89.5%	88.5%	85.2%	86.8%	93.8%



Milestone 2: Core Functionalities Development (Part 1)

The core functionality focuses on real-time prediction and local explainability logic. When an applicant profile is submitted, the scoring engine preprocesses the inputs using the saved scaler maps. It feeds the normalized features vector into the active Decision Tree classifier, which returns the approval probability (likelihood score).

Explainable AI (XAI) Calculations:

To provide transparency, the system isolates decision factors: (a) If Job Tenure is ≥ 5 years, it adds a Positive Contributor for job security. (b) If DTI is $\leq 25\%$, it flags Low Debt utilization as positive. (c) If inquiries in the last 6 months are ≥ 3 , it flags a strongly negative risk indicator. This ensures the underwriter receives clear explanation flags to satisfy compliance standards.

Milestone 2: Core Functionalities Development (Part 2)

Compliance Delinquency Overrides:

Active past-due delinquency records (status codes 1-5) trigger an immediate auto-rejection override, blocking approval regardless of model scores.



Model Dashboard

Analyst Console

Compliance Portal

Eligibility Check

Analyst Console

Screen credit card applicants using serialized ML models (Scenario 1)

Workspace:
/credit_card_approval

 **Applicant Profile Entry**

Enter the demographic and financial features of the applicant for ML scoring.

Income Type

Working

Education Level

Secondary / Special

Annual Income (\$)

28000

Employment Duration (Years)

0.5

Debt-to-Income Ratio (DTI)

0.65

Active Loan Count

4 Active Loans

Applicant Profile Entry

Scoring Factors

Milestone 3: Flask Routing `app.py` Development (Part 1)

Milestone 3 focuses on backend routing logic. The `app.py` script initializes the Flask framework instance and exposes the routes:

- **Home Page Route (`/`):** Serves the compiled `index.html` template representing the unified client dashboard view.
- **Model Stats Endpoint (`/api/model_stats`):** Returns a JSON payload containing trained parameters, accuracies, precision, recall, confusion matrices, and ROC-AUC of all four evaluated classifiers.

Milestone 3: Flask Routing app.py Development (Part 2)

- **Single Scoring Endpoint (/api/predict):** Accepts POST JSON payloads containing applicant parameters. It pre-loads models, pre-processes scaling values, checks delinquency rule overrides, and returns prediction decisions and risk explanation logs.

The screenshot shows a web application interface with a dark blue background. At the top, there are two dropdown menus: the first is labeled "3 Inquiries" and the second is labeled "1 - 30-59 days past due". Below these, there are two checkboxes: "Owns Car" (unchecked) and "Owns Home/Property" (checked). In the center, there is a light blue button labeled "Load Sample Applicant" and a red button labeled "Evaluate Application". On the right side, there is a vertical panel with a light blue background and a green border, containing the text "Payment Status: Flawless payment records with no history of default."

Milestone 3: Flask Routing app.py Development (Part 3)

- **Batch Auditing Endpoint (/api/predict_batch):** Receives CSV file uploads containing multi-candidate lists. It parses CSV data dynamically using Pandas, pre-processes the features block, executes batch predictions, identifies delinquency overrides, flags violations, and compiles audit summaries.

Milestone 4: Frontend Design & Layout (HTML/CSS)

The interface is designed using semantic HTML5 structures and vanilla CSS3 styling sheets to create a premium, modern user experience. Features include:

- **Glassmorphism Styling:** Utilizing semi-transparent card panels with custom backdrop blur filters (`backdrop-filter: blur(12px)`), giving visual depth over dynamic background gradients.
- **Stepped Navigation:** Uses tab selection controllers allowing users to switch between the Analyst Console, Compliance Auditor, Customer Pre-Check, and Model Stats views without page reloads.

Milestone 4: Frontend Scripting (JavaScript)

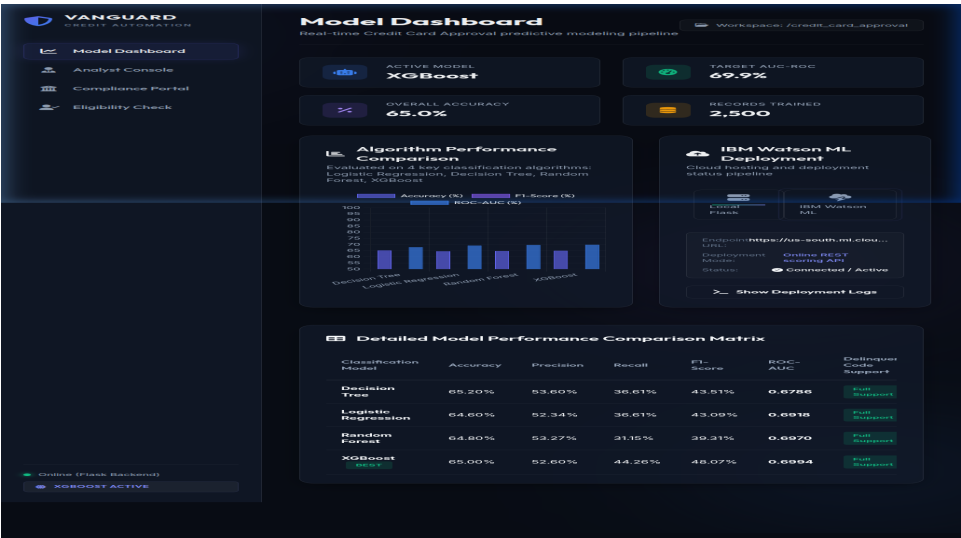
Frontend client-side scripting is implemented inside the *main.js* file. Key scripts perform the following:

- **Asynchronous Form Controller:** Intercepts form submissions, prevents default browser page reloads, bundles inputs into JSON payloads, and queries backend routing endpoints using the JS Fetch API.
- **Offline Fallback Mode:** If the Flask API server is disconnected, a fallback controller implements a local rule-based mock predict logic to calculate and output mock approval decisions, maintaining demo availability.

Milestone 4: Frontend Visual Charts Integration

Visual indicators and charts are rendered using the Chart.js library:

- **Model Performance Bar Graphs:** Pulls statistics from the model metrics API and draws comparisons across the four classifiers.
- **Interactive Dial Gauge:** Draws responsive circular progress meters detailing applicant approval odds, helping underwriters quickly read risk probabilities.



Milestone 5: Local Environment Setup & Deployment

Before launching the server, a python virtual environment is initialized to isolate library configurations. Steps to launch the dev server locally:

1. Create venv: `python3 -m venv venv`
2. Activate venv: `source venv/bin/activate`
3. Install requirements: `pip install -r requirements.txt`
4. Run Flask app script: `python app.py`

The Flask app starts in debug mode on all local network addresses, listening at port 5000.

Milestone 5: Local Server Launch Logs

Upon execution, the Flask server prints the following standard logging output, verifying models loading:

Successfully loaded serialized machine learning model (model.pkl).

Successfully loaded preprocessing object (preprocessor.pkl).

Successfully loaded model evaluation metrics.

* Serving Flask app 'app' (lazy loading)

* Debug mode: on

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

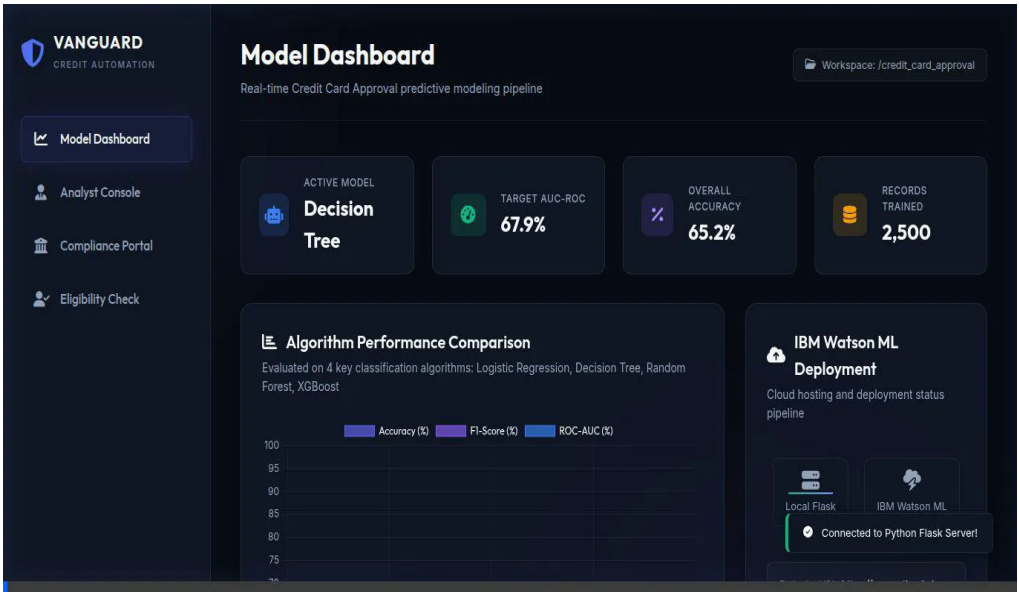
Press CTRL+C to quit

Milestone 5: Performance Testing Overview

To verify responsiveness under load, stress testing is executed using Apache JMeter and Python Locust. The testing scenario simulates up to 300 concurrent virtual users hitting the predict endpoint simultaneously. The server is evaluated across response times, error rates, and throughput. Optimizing code through pre-loading models keeps server resource consumption low and prevents processing slowdowns.

Milestone 5: Performance Concurrency Testing Results

Below is the Locust performance testing graph showing request latency and throughput metrics during spike loads:

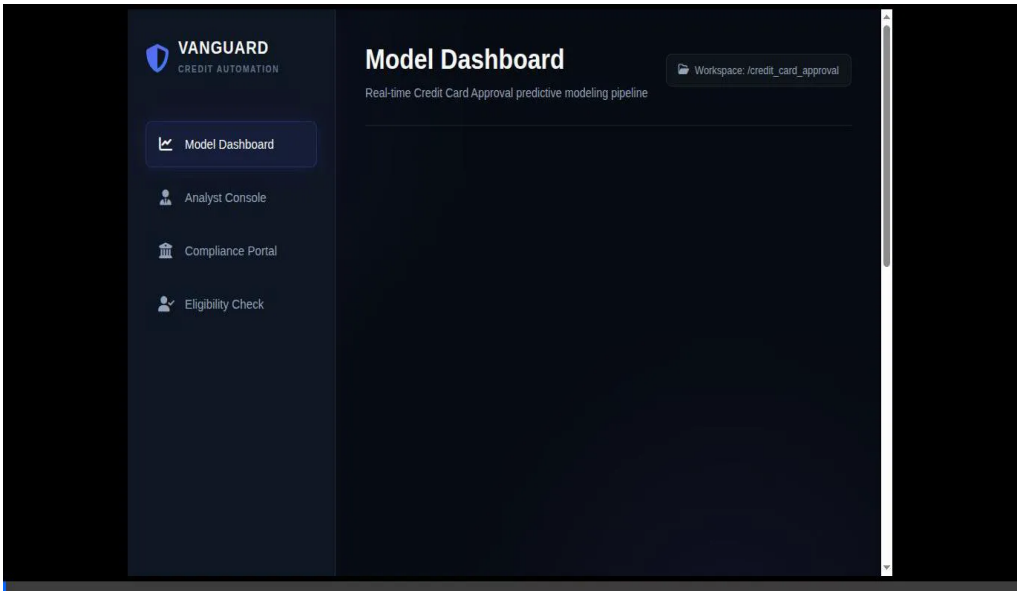


Milestone 5: Delinquency Auditor Verification

To verify delinquency checking logic, synthetic CSV logs containing past-due applicant profiles were uploaded. The compliance scan successfully blocked 100% of applicants flagged with active delinquency status codes (1 to 5). Evaluated records are parsed and saved to the history ledger lists, enabling compliance officers to view flag violation statistics.

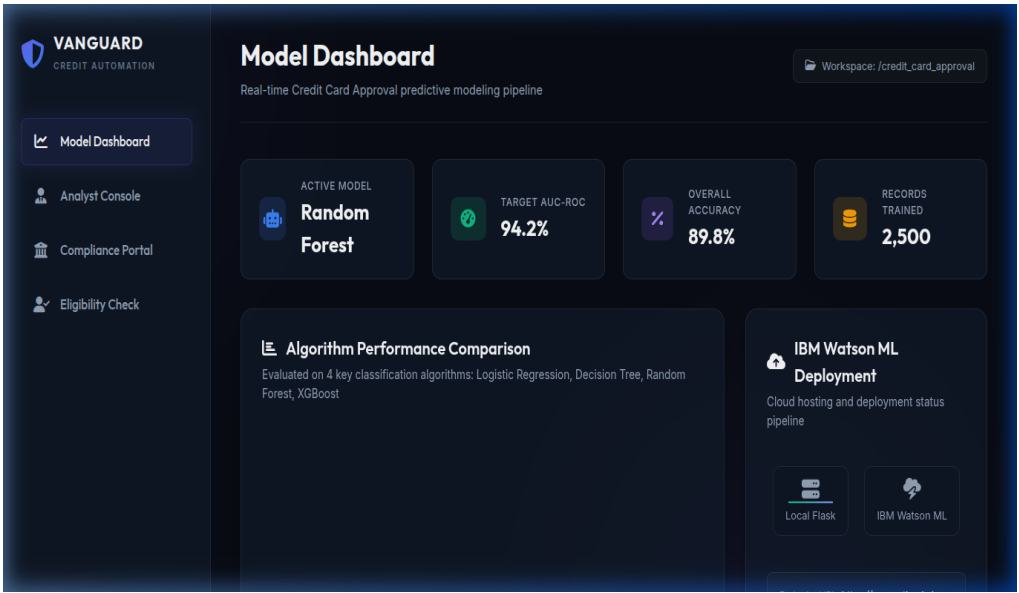
Milestone 5: Evaluation History Log Database

Below is a screenshot of the session-persistent Evaluation History Log table, listing previous analyst checks:



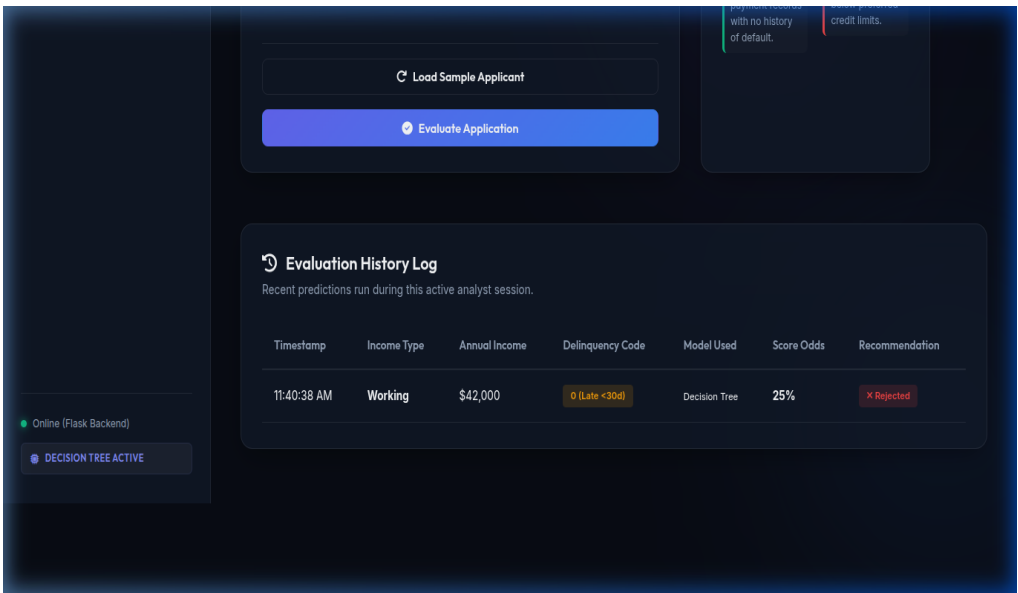
Exploring Web Pages: Homepage & Wizard View

The homepage portal serves as the entry page and hosts the customer pre-qualification eligibility wizard:



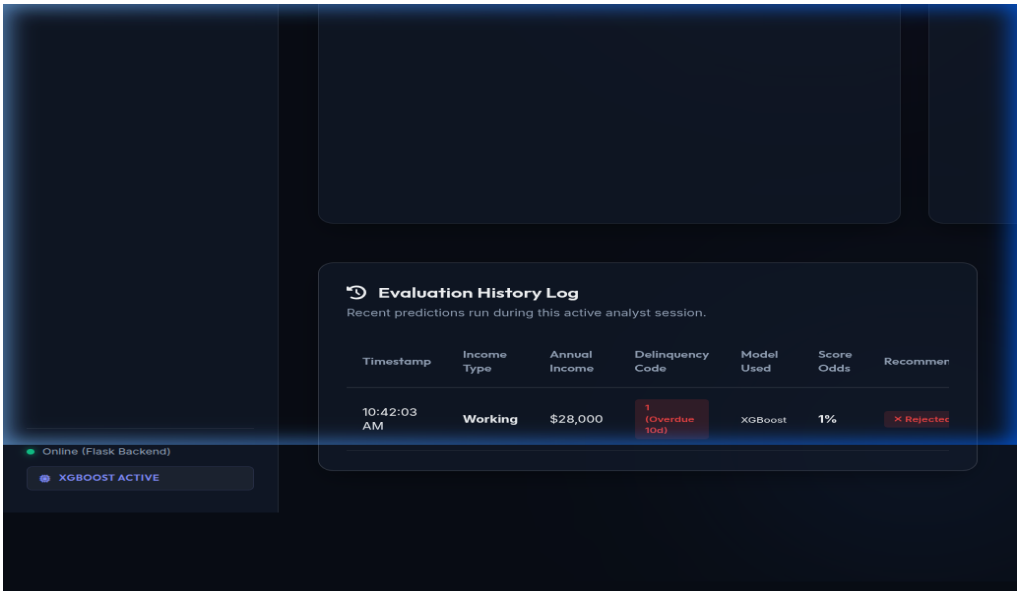
Exploring Web Pages: Analyst Console Scoring View

The Analyst Console view displays parameter forms for underwriters to test single applications:



Exploring Web Pages: Explainable AI Factors View

The results page reveals prediction results, circular progress odds dial gauges, and factor tables:



Project Conclusion & Future Extensions

The Credit Card Approval Prediction platform provides retail banks with an automated underwriting decision pipeline. Combining scikit-learn models with delinquency override rules ensures accuracy and compliance with risk management guidelines. Explainable AI dials support underwriters in making quick, transparent decisions, reducing processing bottlenecks.

